Assignment Report | Daniel De Chalons

§ Introduction

The National Health Service (NHS) is the public health service for the United Kingdom, and therefore is an organisation that is funded by the taxpayer. It is also the largest organisation in the UK, where around 1 in 40 people in England alone work for this organisation.[1] As such, it is arguably one of the most important organisations in the UK and maintaining strong and efficient fiscal responsibility is paramount. So, the need arises to determine service use across the service. The following analysis aims to do that by looking at where appointments are most attended, when, and how many are missed or attended.

§ Process

The first steps I took were to initially have a look at the source files in a text reader, and then inside of excel (excel being a format I know more intimately) 'get a feel' for the data, and understand what it contains. Then, in the Jupyter notebook, I import pandas and numpy, then move into importing the two csv files and one excel file (as in the below screenshot). Following this, I used the df.head() and df.tail() functions to see if the data was ported in properly.

```
# Import the necessary libraries.
import pandas as pd
import numpy as np
```

```
# Give a name to the dataframe that makes sense and is easy to refer back to.
duration = pd.read_csv('actual_duration.csv')

# Print the dataframe; viewing the first and last sections of the data for a sense-check.
print(duration.head())

print(duration.tail())
```

Once it was clear the data had ported correctly, the next step was to make sure there were no null values in the columns and then determine the makeup of the data frame (df herein). I used the df.isnull().values.any() because I saw this as a sensible first step in determining if there were any nulls *at all* in the dataset, where a boolean value TRUE is returned if there were nulls present (and vice versa). If TRUE was returned I could then use df.isnull().sum() to take a closer look at where and in what number the nulls were in the df.
The second block in the screenshot below allowed me to see the size of the df by returning no. of rows, no. of columns; df.dtypes() returned which value was stored for each column.
To get the descriptive statistics of the df, .describe() was used. This returns those main statistics of the numeric column in the set.

```
# Check if there are any null values at all in the dataset as a broad search.
duration.isnull().values.any()
```

```
# Get a description of the data to better understand it
print(duration.shape)

print(duration.dtypes)
```

```
# Further describe the data by getting the descriptive statistics
duration.describe()
```

To answer the question "What is the number of locations, service settings, context types, national categories, and appointment statuses in the data sets?" I ran the .nunique() function so that a simple count number was returned for each of the columns (as below).

```
# No. of Locations. Determine this by running a unique count on the sub-icb location name column
print(categories['sub_icb_location_name'].nunique())
```

```
106
```

Moving further into the analysis, the below code block looks at the question "What is the date range of the provided data sets, and which service settings reported the most appointments for a

[1] https://www.stepintothenhs.nhs.uk/about-the-nhs

specific period?". I initially changed the data type of each respective date column to the 'datetime' type, as from the above sense check they were returned as 'object' values. It ensures the right value is returned when I run the min() and max() functions on the dates to see the earliest and latest for each respective set.

```
# First convert the date column to the proper format so that it reads properly.
# Specifying the format stops the potential error for individual parsing of dates.
duration['appointment_date'] = pd.to_datetime(duration['appointment_date'], format = '%d-%b-%y')

# Next set.
regional['appointment_month'] = pd.to_datetime(regional['appointment_month'], format = '%Y-%m')

# Next set.
categories['appointment_date'] = pd.to_datetime(categories['appointment_date'], format = '%d-%m-%Y')

# Set the variables for the earliest and latest dates for all the data sets.
duration_min_date = duration['appointment_date'].min()
duration_max_date = duration['appointment_date'].max()

# Next set.
regional_min_date = regional['appointment_month'].min()
regional_max_date = regional['appointment_month'].max()

# Next set.
categories_min_date = categories['appointment_date'].min()
categories_max_date = categories['appointment_date'].max()

# Print the variables with text in a digestible way.
print('In the Actual Duration file:\nThe earliest date is: ', duration_min_date)
print('The latest date is: ', duration_max_date)
print('\nIn the appointments regional file:\nThe earliest date is: ', regional_min_date)
print('The latest date is: ', regional_max_date)
print('\nIn the National Categories file:\nThe earliest date is: ', categories_min_date)
print('The latest date is: ', categories_max_date)
```

§ Visuals

Moving forward to answering the question "What monthly and seasonal trends are evident, based on the number of appointments for service settings, context types, and national categories?" I decided a table and a line plot visual would be best paired, as there are many categories for comparison. Below is a snippet of the output and code to show the table of values when looking at the number of appointments by service setting. I grouped the values by service setting and then by the appointment month. I wanted the viewer to look at the numbers by group and a high-to-low appointment count so all the numbers could be looked at group by group and not a mix of values, which would make it harder to compare unless it was written down separately, slowing down process. So, I used the .sort_values() function on the created df to achieve this. Finally, to improve readability on the returned values I used .style.format() to add commas into the number, otherwise it is harder to gauge the number on first inspection.

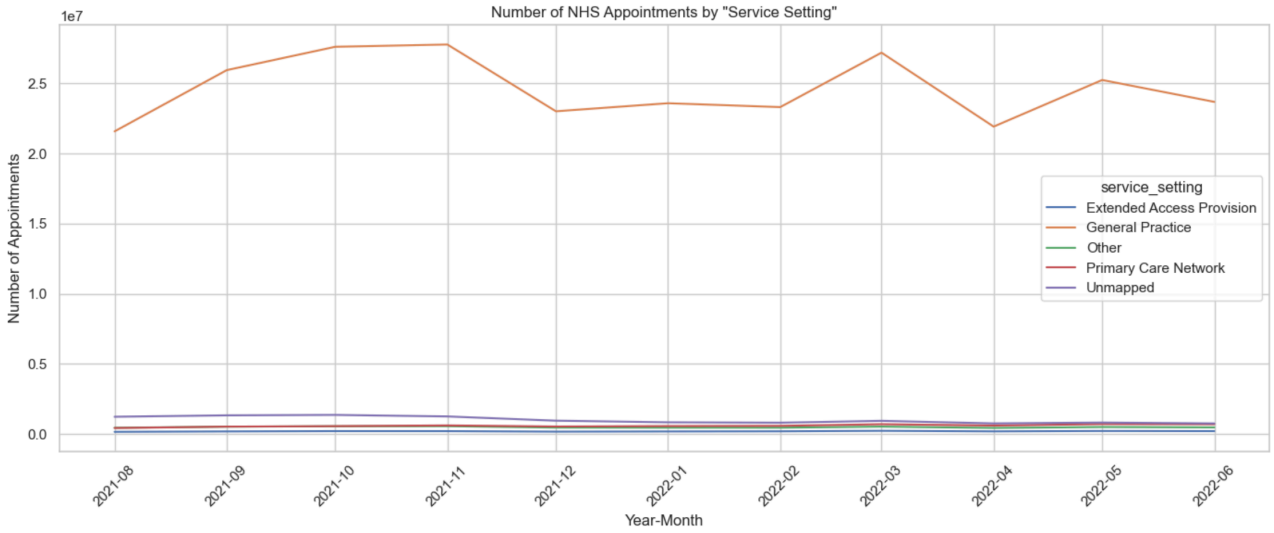
```
import seaborn as sns
import matplotlib.pyplot as plt

# Group by service setting and month then sum the appointment count.
monthly_counts = categories.groupby(['service_setting', 'appointment_month'])['count_of_appointments'].sum().reset_index()

# Sort the values so that the values show by service setting and from high to low.
monthly_counts = monthly_counts.sort_values(by = ['service_setting', 'count_of_appointments'], ascending = [True, False])

# Aid readability by formatting with commas.
formatted_monthly_count = monthly_counts.style.format({'count_of_appointments': '{:,}'})

formatted_monthly_count
```

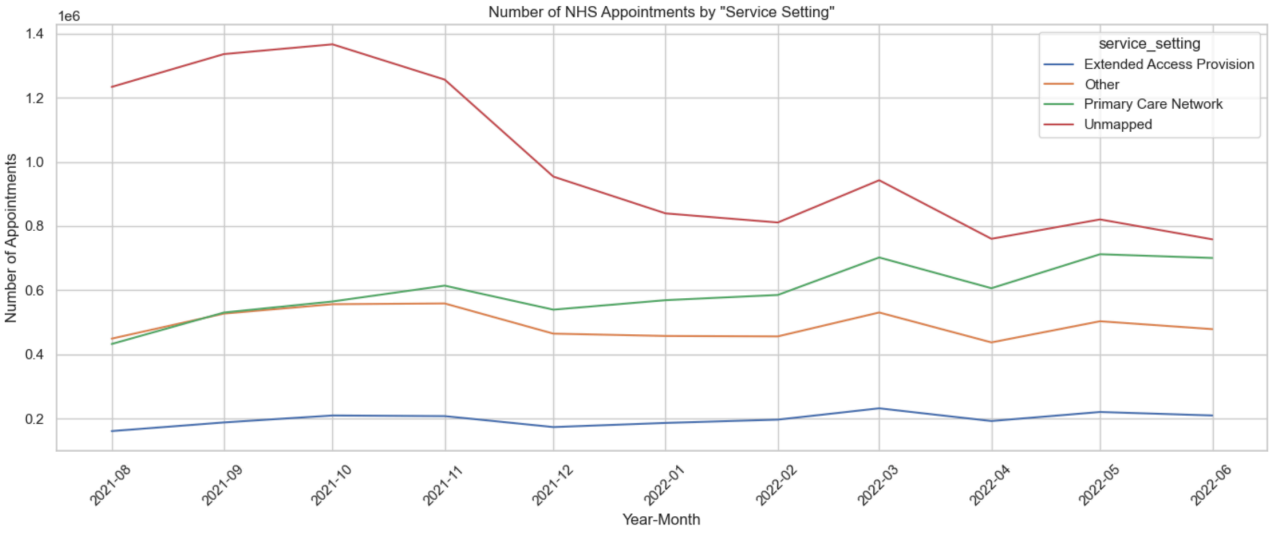
	service_setting	appointment_month	count_of_appointments
7	Extended Access Provision	2022-03	231,905
9	Extended Access Provision	2022-05	220,511
10	Extended Access Provision	2022-06	209,652

For the below code for the line-plot graph, I added plt.xticks() and plt.tight_layout() to the formatting. Although for this graph their impact on the visual is minimal, it can serve to aid readability for tighter graphs and ensure there is no overlap on values and so can be considered a precautionary measure.

```
# Make sure that appointment_month is in datetime format (This was not changed in the above date check)
monthly_counts['appointment_month'] = pd.to_datetime(monthly_counts['appointment_month'], format='%Y-%m')

# Set the plot style
sns.set(style = 'whitegrid')

# Create the line plot
plt.figure(figsize = (14, 6))
sns.lineplot(
    data = monthly_counts,
    x = 'appointment_month',
    y = 'count_of_appointments',
    hue = 'service_setting')

# Format and Label
plt.title('Number of NHS Appointments by "Service Setting"')
plt.xlabel('Year-Month')
plt.ylabel('Number of Appointments')
plt.xticks(rotation=45)
plt.tight_layout()

plt.show()
```

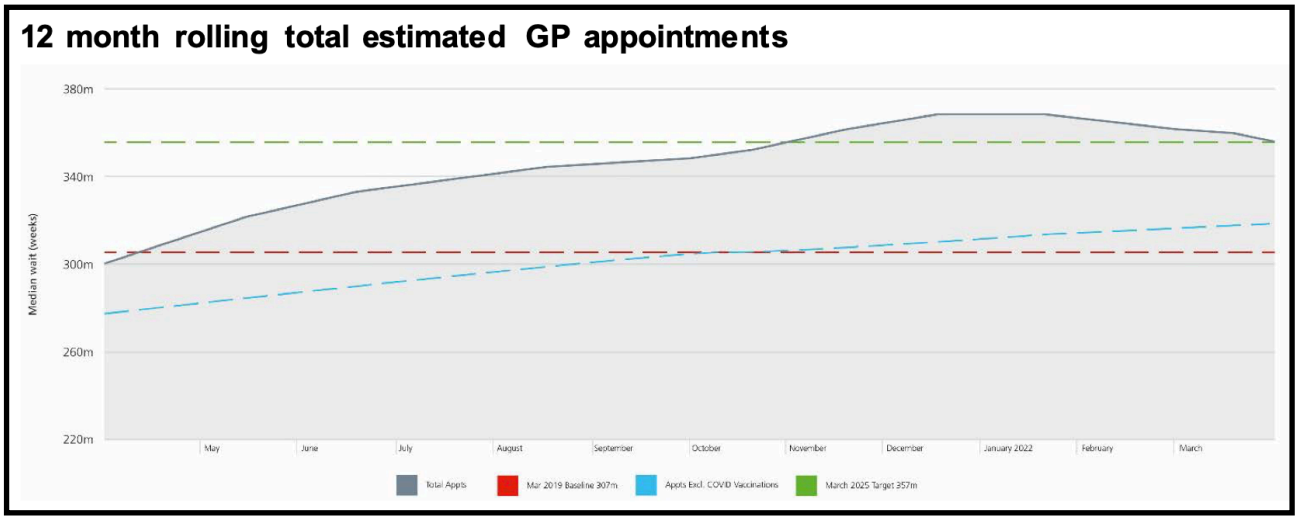
The output for the visuals is as below. We can easily compare the service settings against the number of appointments on the y axis. Although, an issue arised where as the 'General Practice' category had so many more appointments, the output was skewed. Hence, I decided to omit this value from a secondary graph to the same question, so as to get a closer look (below initial graph)





§ Insights

One of the first and most obvious patterns in the data is the spike in the number of appointments across all categories in March. If we look at the illustration below (an estimate of the GP appointments over the year 2021-2022),[2] we can see that the prediction for March is actually a downturn in appointments. The HFMA's report states "…Unspent allocation…cannot be carried forward to the next financial year"[3], so, what could be drawn from this is that the estimate may reflect *actual* demand, but not real appointment distribution (GP referral), where practises are referring more patients in an attempt to make budget by year end.



Another inference can be drawn from the attendance data. The analysis showed that highest actual utilisation of resource (in no. of appointments) for one month was 30,405,070 in November 2021. Given that the NHS has on average a capacity of 1.2m appointments per day,[4] November would have capacity of 36m and therefore, would have not exceeded capacity, showing adequate staffing levels.

To conclude, it would be sensible to take a more granular day-by-day look at the data to see *daily* utilisation, and also which specific locations see the most strain so that the service is best informed on where to allocate funding or increase/decrease resource. Also, the NHS might look to match predictions against actual seasonal spikes, such as fiscal year end, to refine their own predictions models.

---

[2] https://assets.publishing.service.gov.uk/media/63d2aa7b8fa8f519db6e912c/nhs-england-annual-report-and-accounts-2021-to-2022-print.pdf?

[3] https://www.hfma.org.uk/publications/review-nhs-capital-financing-regime?

[4] https://fourthrev.instructure.com/courses/895/pages/assignment-activity-6-making-recommendations?module_item_id=67564